Robust Finite-Difference Greeks with Richardson Extrapolation

Motivation

The accurate computation of option Greeks—sensitivity measures such as Delta, Gamma, Theta, Vega, and Rho—is fundamental to quantitative finance, risk management, and derivatives trading. These sensitivities quantify how option prices respond to changes in underlying market parameters, enabling portfolio hedging, risk assessment, and regulatory capital calculations. However, computing Greeks with sufficient accuracy whilst maintaining computational efficiency presents significant numerical challenges.

Traditional finite-difference methods for Greek computation suffer from discretisation errors that arise from the choice of perturbation step size. Too large a step size introduces truncation errors, whilst too small a step size amplifies numerical round-off errors, particularly when prices are computed via Monte Carlo simulation or partial differential equation (PDE) solvers. This fundamental trade-off between truncation and round-off errors creates a delicate optimisation problem that varies across different market conditions, option types, and numerical methods.

Richardson extrapolation provides an elegant solution to this problem by combining multiple finite-difference approximations computed with different step sizes to eliminate leading-order error terms, thereby achieving higher-order accuracy without requiring prohibitively small step sizes. By systematically refining step sizes and extrapolating to the limit, Richardson extrapolation can transform first-order or second-order finite-difference approximations into significantly more accurate estimates, making it an essential tool for production-grade risk systems.

This project implements a comprehensive framework for robust Greek computation that combines adaptive step sizing, Richardson extrapolation, and central differencing optimisation. The system automatically determines optimal perturbation sizes based on price grid characteristics, applies Richardson extrapolation to enhance accuracy, and provides detailed convergence diagnostics. The framework supports both PDE-based pricing (via finite-difference grids) and Monte Carlo pricing methods, making it applicable across a wide range of derivative products and numerical pricing engines.

By integrating rigorous numerical analysis with interactive visualisation capabilities, this tool enables quantitative analysts, risk managers, and traders to compute reliable Greeks efficiently, understand convergence behaviour, and validate results across different market scenarios and numerical configurations.

Theory

This project implements a robust framework for computing option Greeks using finite-difference methods enhanced with Richardson extrapolation and adaptive step sizing.

Option Greeks

Definition (Option Greeks)

Option Greeks are partial derivatives of the option price $V(S, t, \sigma, r)$ with respect to various market parameters:

$$\Delta = \frac{\partial V}{\partial S} \quad \text{(Delta)}$$
$$\Gamma = \frac{\partial^2 V}{\partial S^2} \quad \text{(Gamma)}$$
$$\Theta = -\frac{\partial V}{\partial t} \quad \text{(Theta)}$$
$$\nu = \frac{\partial V}{\partial \sigma} \quad \text{(Vega)}$$
$$\rho = \frac{\partial V}{\partial r} \quad \text{(Rho)}$$

where S is the underlying asset price, t is time, σ is volatility, and r is the risk-free interest rate.

These sensitivities play crucial roles in portfolio management: Delta measures price sensitivity to underlying movements and guides delta hedging strategies; Gamma quantifies the rate of change of Delta and indicates hedging frequency requirements; Theta captures time decay; Vega measures volatility sensitivity; and Rho quantifies interest rate exposure.

Finite-Difference Approximation

Finite-difference methods approximate derivatives by evaluating the pricing function at perturbed parameter values. For a first-order derivative, the forward difference approximation is:

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x)}{h},$$

whilst the backward difference is:

$$\frac{\partial V}{\partial x} \approx \frac{V(x) - V(x-h)}{h},$$

where h is the perturbation step size. The central difference approximation, which typically provides better accuracy, is:

$$\frac{\partial V}{\partial x} \approx \frac{V(x+h) - V(x-h)}{2h}.$$

For second-order derivatives, the central difference approximation is:

$$\frac{\partial^2 V}{\partial x^2} \approx \frac{V(x+h) - 2V(x) + V(x-h)}{h^2}.$$

Truncation and Round-Off Errors

Theorem (Error Analysis)

The total error in finite-difference approximation consists of truncation error and round-off error:

$$\text{Total Error} = \underbrace{\mathcal{O}(h^p)}_{\text{Truncation}} + \underbrace{\frac{\epsilon}{h}}_{\text{Round-off}},$$

where p is the order of the method ($p = 1$ for forward/backward differences, $p = 2$ for central differences) and ϵ is the machine precision or pricing error.

The truncation error decreases as $h \to 0$, but the round-off error increases inversely with h. The optimal step size h^* minimises the total error, typically satisfying:

$$h^* \propto \epsilon^{1/(p+1)}.$$

For central differences with $p = 2$ and typical pricing errors $\epsilon \approx 10^{-6}$, the optimal step size is approximately $h^* \approx 10^{-2}$.

Richardson Extrapolation

Definition (Richardson Extrapolation)

Richardson extrapolation combines approximations computed with different step sizes to eliminate leading-order error terms. If $F(h)$ is an approximation with error $\mathcal{O}(h^p)$, then:

$$F_{\text{extrapolated}} = \frac{2^p F(h/2) - F(h)}{2^p - 1}$$

eliminates the $\mathcal{O}(h^p)$ error term, yielding an approximation with error $\mathcal{O}(h^{p+1})$.

For central differences with $p = 2$, Richardson extrapolation gives:

$$\frac{\partial V}{\partial x} \approx \frac{4 \cdot \frac{V(x+h/2)-V(x-h/2)}{h} - \frac{V(x+h)-V(x-h)}{2h}}{3} = \frac{-V(x+h) + 8V(x+h/2) - 8V(x-h/2) + V(x-h)}{6h}.$$

This five-point stencil achieves fourth-order accuracy $\mathcal{O}(h^4)$ compared to the second-order accuracy $\mathcal{O}(h^2)$ of standard central differences.

Adaptive Step Sizing

Adaptive step sizing algorithms automatically determine optimal perturbation sizes by:

1. **Initial estimate**: Start with a step size based on parameter magnitude, e.g., $h_0 = \alpha \cdot |x|$ where $\alpha \approx 0.01$ for asset prices.
2. **Convergence testing**: Compute approximations with step sizes h, $h/2$, and $h/4$, and check if Richardson-extrapolated values converge.
3. **Error estimation**: Estimate numerical error by comparing approximations at different refinement levels.
4. **Step adjustment**: If error exceeds tolerance, refine step size; if error is much smaller than tolerance, coarsen step size to reduce computational cost.

The adaptive algorithm ensures that step sizes are neither too large (causing truncation errors) nor too small (amplifying round-off errors), whilst maintaining computational efficiency.

Central Differencing Optimisation

For second-order derivatives like Gamma, the standard central difference formula is:

$$\Gamma = \frac{V(S+h) - 2V(S) + V(S-h)}{h^2}.$$

With Richardson extrapolation using step sizes h and $h/2$:

$$\Gamma_{\text{extrapolated}} = \frac{4\Gamma(h/2) - \Gamma(h)}{3} = \frac{16V(S+h/2) - 32V(S) + 16V(S-h/2) - V(S+h) + 2V(S) - V(S-h)}{3h^2}.$$

This achieves fourth-order accuracy and significantly improves robustness, particularly when prices are computed via Monte Carlo simulation with inherent statistical noise.

Greeks from Price Grids

When option prices are computed on a grid (e.g., from PDE solvers or Monte Carlo with bucketing), Greeks can be computed directly from grid values using finite differences:

- **Delta**: $\Delta \approx \frac{V_{i+1} - V_{i-1}}{S_{i+1} - S_{i-1}}$ for grid points S_i.
- **Gamma**: $\Gamma \approx \frac{V_{i+1} - 2V_i + V_{i-1}}{(\Delta S)^2}$ where $\Delta S = S_{i+1} - S_i$.
- **Theta**: $\Theta \approx -\frac{V^{t+\Delta t} - V^t}{\Delta t}$ for time grid points.

Richardson extrapolation can be applied at each grid point by using neighbouring grid values at different refinement levels, provided the grid structure supports such refinement.

Monte Carlo Greeks

For Monte Carlo pricing, Greeks are computed by:

1. **Perturbation**: Generate price estimates $V(x)$, $V(x+h)$, and $V(x-h)$ using the same random seed to ensure pathwise consistency.
2. **Finite difference**: Apply central difference formula to Monte Carlo price estimates.
3. **Richardson extrapolation**: Combine estimates from different step sizes to reduce both discretisation and Monte Carlo sampling errors.
4. **Error propagation**: Account for Monte Carlo standard errors in Greek estimates using error propagation formulas.

The use of common random numbers (same seed for all perturbations) is crucial for reducing variance in Greek estimates, as it ensures that differences in prices arise from parameter changes rather than random variation.

Error Estimation and Convergence Diagnostics

For a Richardson-extrapolated Greek estimate $\hat{G}$ computed from approximations $G(h)$ and $G(h/2)$, the error can be estimated as:

$$\text{Estimated Error} = \left|\hat{G} - G(h/2)\right|.$$

Convergence is verified by checking that:

$$|G(h) - G(h/2)| \approx 2^p \left|G(h/2) - G(h/4)\right|,$$

which confirms that the error decreases at the expected rate. If this relationship fails, it may indicate that step sizes are too small (round-off errors dominate) or that the pricing function lacks sufficient smoothness.

Data Requirements

- **Option pricing function**: A callable function or method that computes option prices $V(S, t, \sigma, r, \ldots)$ for given parameter values. This may be a PDE solver, Monte Carlo engine, analytical formula, or any numerical pricing method.
- **Base parameter values**: Initial asset price S_0, time to maturity T, volatility σ, risk-free rate r, strike price K, and other relevant parameters (dividend yield, barrier levels, etc.).
- **Price grids (optional)**: Pre-computed option prices on grids of (S, t) values from PDE solvers or Monte Carlo bucketing, enabling direct grid-based Greek computation.
- **Convergence tolerances**: Maximum acceptable error in Greek estimates, typically specified as absolute or relative tolerances (e.g., 10^{-4} or 0.01%).
- **Step size bounds**: Minimum and maximum allowed perturbation sizes to prevent numerical instability or excessive computation.
- **Monte Carlo parameters (if applicable)**: Number of paths, random seed for common random numbers, convergence criteria for price estimates.
- **Grid specifications (if applicable)**: Grid dimensions, spacing, boundary conditions for PDE-based pricing.

The tool includes synthetic pricing functions and parameter sets for demonstration purposes, enabling testing and validation without requiring external pricing engines.

Code

The following code excerpts illustrate production-quality implementations using object-oriented design principles.

Python: Finite-Difference Greeks with Richardson Extrapolation

```
import numpy as np
import pandas as pd
from typing import Callable, Dict, Tuple, Optional, List
from scipy.optimize import minimize_scalar
import warnings

```

```python
class FiniteDifferenceGreeks:
    def __init__(self, pricing_function: Callable,
                 base_params: Dict[str, float],
                 tolerance: float = 1e-4,
                 min_step: float = 1e-6,
                 max_step: float = 0.1):
        self.pricing_function = pricing_function
        self.base_params = base_params.copy()
        self.tolerance = tolerance
        self.min_step = min_step
        self.max_step = max_step

    def compute_delta(self, S: float, step_size: Optional[float] = None,
                      use_richardson: bool = True) -> Tuple[float, float]:
        if step_size is None:
            step_size = self._adaptive_step_size(S, 'delta')

        h = step_size
        V0 = self.pricing_function({**self.base_params, 'S': S})

        if use_richardson:
            V_plus_h = self.pricing_function({**self.base_params, 'S': S + h})
            V_minus_h = self.pricing_function({**self.base_params, 'S': S - h})
            delta_h = (V_plus_h - V_minus_h) / (2 * h)

            V_plus_h2 = self.pricing_function({**self.base_params, 'S': S + h/2})
            V_minus_h2 = self.pricing_function({**self.base_params, 'S': S - h/2})
            delta_h2 = (V_plus_h2 - V_minus_h2) / h

            delta_richardson = (4 * delta_h2 - delta_h) / 3
            error_estimate = abs(delta_richardson - delta_h2)

            return delta_richardson, error_estimate
        else:
            V_plus_h = self.pricing_function({**self.base_params, 'S': S + h})
            V_minus_h = self.pricing_function({**self.base_params, 'S': S - h})
            delta = (V_plus_h - V_minus_h) / (2 * h)
            return delta, 0.0

    def compute_gamma(self, S: float, step_size: Optional[float] = None,
                      use_richardson: bool = True) -> Tuple[float, float]:
        if step_size is None:
            step_size = self._adaptive_step_size(S, 'gamma')

        h = step_size
        V0 = self.pricing_function({**self.base_params, 'S': S})

        if use_richardson:
```

```
            V_plus_h = self.pricing_function({**self.base_params, 'S': S + h})
            V_minus_h = self.pricing_function({**self.base_params, 'S': S - h})
            gamma_h = (V_plus_h - 2 * V0 + V_minus_h) / (h ** 2)

            V_plus_h2 = self.pricing_function({**self.base_params, 'S': S + h/2})
            V_minus_h2 = self.pricing_function({**self.base_params, 'S': S - h/2})
            gamma_h2 = (V_plus_h2 - 2 * V0 + V_minus_h2) / ((h/2) ** 2)

            gamma_richardson = (4 * gamma_h2 - gamma_h) / 3
            error_estimate = abs(gamma_richardson - gamma_h2)

            return gamma_richardson, error_estimate
        else:
            V_plus_h = self.pricing_function({**self.base_params, 'S': S + h})
            V_minus_h = self.pricing_function({**self.base_params, 'S': S - h})
            gamma = (V_plus_h - 2 * V0 + V_minus_h) / (h ** 2)
            return gamma, 0.0

    def compute_theta(self, t: float, step_size: Optional[float] = None,
                      use_richardson: bool = True) -> Tuple[float, float]:
        if step_size is None:
            step_size = self._adaptive_step_size(t, 'theta')

        h = step_size
        V0 = self.pricing_function({**self.base_params, 't': t})

        if use_richardson:
            V_plus_h = self.pricing_function({**self.base_params, 't': t + h})
            V_minus_h = self.pricing_function({**self.base_params, 't': t - h})
            theta_h = -(V_plus_h - V_minus_h) / (2 * h)

            V_plus_h2 = self.pricing_function({**self.base_params, 't': t + h/2})
            V_minus_h2 = self.pricing_function({**self.base_params, 't': t - h/2})
            theta_h2 = -(V_plus_h2 - V_minus_h2) / h

            theta_richardson = (4 * theta_h2 - theta_h) / 3
            error_estimate = abs(theta_richardson - theta_h2)

            return theta_richardson, error_estimate
        else:
            V_plus_h = self.pricing_function({**self.base_params, 't': t + h})
            theta = -(V_plus_h - V0) / h
            return theta, 0.0

```

```
    def compute_vega(self, sigma: float, step_size: Optional[float] =
        None,
                     use_richardson: bool = True) -> Tuple[float, float
                        ]:
        if step_size is None:
            step_size = self._adaptive_step_size(sigma, 'vega')

        h = step_size
        V0 = self.pricing_function({**self.base_params, 'sigma': sigma
           })

        if use_richardson:
            V_plus_h = self.pricing_function({**self.base_params, '
               sigma': sigma + h})
            V_minus_h = self.pricing_function({**self.base_params, '
               sigma': sigma - h})
            vega_h = (V_plus_h - V_minus_h) / (2 * h)

            V_plus_h2 = self.pricing_function({**self.base_params, '
               sigma': sigma + h/2})
            V_minus_h2 = self.pricing_function({**self.base_params, '
               sigma': sigma - h/2})
            vega_h2 = (V_plus_h2 - V_minus_h2) / h

            vega_richardson = (4 * vega_h2 - vega_h) / 3
            error_estimate = abs(vega_richardson - vega_h2)

            return vega_richardson, error_estimate
        else:
            V_plus_h = self.pricing_function({**self.base_params, '
               sigma': sigma + h})
            V_minus_h = self.pricing_function({**self.base_params, '
               sigma': sigma - h})
            vega = (V_plus_h - V_minus_h) / (2 * h)
            return vega, 0.0

    def compute_rho(self, r: float, step_size: Optional[float] = None,
                    use_richardson: bool = True) -> Tuple[float, float]:
        if step_size is None:
            step_size = self._adaptive_step_size(r, 'rho')

        h = step_size
        V0 = self.pricing_function({**self.base_params, 'r': r})

        if use_richardson:
            V_plus_h = self.pricing_function({**self.base_params, 'r':
               r + h})
            V_minus_h = self.pricing_function({**self.base_params, 'r':
                r - h})
            rho_h = (V_plus_h - V_minus_h) / (2 * h)

            V_plus_h2 = self.pricing_function({**self.base_params, 'r':
                r + h/2})
            V_minus_h2 = self.pricing_function({**self.base_params, 'r'
               : r - h/2})
            rho_h2 = (V_plus_h2 - V_minus_h2) / h

            rho_richardson = (4 * rho_h2 - rho_h) / 3
```

```
            error_estimate = abs(rho_richardson - rho_h2)

            return rho_richardson, error_estimate
        else:
            V_plus_h = self.pricing_function({**self.base_params, 'r': r + h})
            V_minus_h = self.pricing_function({**self.base_params, 'r': r - h})
            rho = (V_plus_h - V_minus_h) / (2 * h)
            return rho, 0.0

    def compute_all_greeks(self, S: float, t: float, sigma: float, r: float,
                           use_richardson: bool = True) -> Dict[str, Tuple[float, float]]:
        delta, delta_err = self.compute_delta(S, use_richardson=use_richardson)
        gamma, gamma_err = self.compute_gamma(S, use_richardson=use_richardson)
        theta, theta_err = self.compute_theta(t, use_richardson=use_richardson)
        vega, vega_err = self.compute_vega(sigma, use_richardson=use_richardson)
        rho, rho_err = self.compute_rho(r, use_richardson=use_richardson)

        return {
            'Delta': (delta, delta_err),
            'Gamma': (gamma, gamma_err),
            'Theta': (theta, theta_err),
            'Vega': (vega, vega_err),
            'Rho': (rho, rho_err)
        }

    def _adaptive_step_size(self, param_value: float, greek_type: str) -> float:
        if abs(param_value) < 1e-10:
            return self.max_step

        initial_step = 0.01 * abs(param_value)
        initial_step = np.clip(initial_step, self.min_step, self.max_step)

        h = initial_step
        converged = False
        max_iterations = 10

        for _ in range(max_iterations):
            if greek_type == 'delta':
                g_h, _ = self.compute_delta(param_value, h, use_richardson=False)
                g_h2, _ = self.compute_delta(param_value, h/2, use_richardson=False)
            elif greek_type == 'gamma':
                g_h, _ = self.compute_gamma(param_value, h, use_richardson=False)
                g_h2, _ = self.compute_gamma(param_value, h/2, use_richardson=False)
```

```
            else:
                return initial_step

            error_estimate = abs(g_h - g_h2)

            if error_estimate < self.tolerance:
                converged = True
                break

            if error_estimate > 10 * self.tolerance:
                h = h / 2
            else:
                h = h * 0.8

        return np.clip(h, self.min_step, self.max_step)

    def compute_greeks_from_grid(self, price_grid: np.ndarray,
                                 S_grid: np.ndarray, t_grid: Optional[
                                    np.ndarray] = None,
                                 use_richardson: bool = True) -> Dict[
                                    str, np.ndarray]:
        n = len(S_grid)
        delta_grid = np.zeros(n)
        gamma_grid = np.zeros(n)

        for i in range(1, n - 1):
            h = S_grid[i + 1] - S_grid[i]
            h_prev = S_grid[i] - S_grid[i - 1]

            if use_richardson and i > 1 and i < n - 2:
                delta_fine = (price_grid[i + 1] - price_grid[i - 1]) /
                    (h + h_prev)
                delta_coarse = (price_grid[i + 2] - price_grid[i - 2])
                    / (2 * (h + h_prev))
                delta_grid[i] = (4 * delta_fine - delta_coarse) / 3
            else:
                delta_grid[i] = (price_grid[i + 1] - price_grid[i - 1])
                     / (h + h_prev)

            if use_richardson and i > 1 and i < n - 2:
                gamma_fine = (price_grid[i + 1] - 2 * price_grid[i] +
                    price_grid[i - 1]) / (h ** 2)
                gamma_coarse = (price_grid[i + 2] - 2 * price_grid[i] +
                     price_grid[i - 2]) / ((2 * h) ** 2)
                gamma_grid[i] = (4 * gamma_fine - gamma_coarse) / 3
            else:
                gamma_grid[i] = (price_grid[i + 1] - 2 * price_grid[i]
                    + price_grid[i - 1]) / (h ** 2)

        result = {'Delta': delta_grid, 'Gamma': gamma_grid}

        if t_grid is not None and len(t_grid) > 1:
            theta_grid = np.zeros(n)
            dt = t_grid[1] - t_grid[0] if len(t_grid) > 1 else 1.0
            for i in range(n):
                if i < len(price_grid) - 1:
                    theta_grid[i] = -(price_grid[i + 1] - price_grid[i
                        ]) / dt
```

```
            result['Theta'] = theta_grid

        return result
```

This implementation provides robust Greek computation with automatic step sizing and Richardson extrapolation, suitable for both analytical and numerical pricing functions.

Python: Streamlit Interactive Dashboard

```
import streamlit as st
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from finite_difference_greeks import FiniteDifferenceGreeks
from scipy.stats import norm

def black_scholes_call(S: float, K: float, T: float, r: float, sigma: float) -> float:
    if T <= 0:
        return max(S - K, 0)
    d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def pricing_function(params: dict) -> float:
    return black_scholes_call(
        S=params.get('S', 100),
        K=params.get('K', 100),
        T=params.get('T', 1.0),
        r=params.get('r', 0.05),
        sigma=params.get('sigma', 0.2)
    )

st.set_page_config(
    page_title="Finite-Difference Greeks Dashboard",
    page_icon="      ",
    layout="wide"
)

st.title("Robust Finite-Difference Greeks with Richardson Extrapolation")

with st.sidebar:
    st.header("Option Parameters")
    S = st.number_input("Spot Price (S)", 50.0, 200.0, 100.0, 1.0)
    K = st.number_input("Strike Price (K)", 50.0, 200.0, 100.0, 1.0)
    T = st.slider("Time to Maturity (years)", 0.01, 2.0, 1.0, 0.01)
    r = st.slider("Risk-Free Rate (%)", 0.0, 10.0, 5.0, 0.1) / 100
    sigma = st.slider("Volatility (%)", 5.0, 50.0, 20.0, 1.0) / 100

    st.header("Greek Computation Settings")
    use_richardson = st.checkbox("Use Richardson Extrapolation", value=True)
    tolerance = st.slider("Convergence Tolerance", 1e-6, 1e-2, 1e-4, format="%.0e")
```

```
    min_step = st.number_input("Min Step Size", 1e-8, 1e-3, 1e-6,
        format="%.0e")
    max_step = st.number_input("Max Step Size", 1e-4, 0.5, 0.1, format=
        "%.3f")

    st.header("Visualisation Settings")
    S_range_min = st.number_input("S Range Min", 50.0, 150.0, 50.0,
        1.0)
    S_range_max = st.number_input("S Range Max", 50.0, 200.0, 150.0,
        1.0)
    n_points = st.slider("Number of Points", 50, 500, 200, 10)

base_params = {'S': S, 'K': K, 'T': T, 'r': r, 'sigma': sigma}
greek_calculator = FiniteDifferenceGreeks(
    pricing_function=pricing_function,
    base_params=base_params,
    tolerance=tolerance,
    min_step=min_step,
    max_step=max_step
)

if st.button("Compute Greeks", type="primary"):
    with st.spinner("Computing Greeks..."):
        greeks = greek_calculator.compute_all_greeks(
            S=S, t=T, sigma=sigma, r=r, use_richardson=use_richardson
        )

        option_price = pricing_function(base_params)

        st.subheader("Greek Values")
        col1, col2, col3, col4, col5 = st.columns(5)

        with col1:
            st.metric("Delta", f"{greeks['Delta'][0]:.6f}",
                      delta=f"Error: {greeks['Delta'][1]:.2e}")
        with col2:
            st.metric("Gamma", f"{greeks['Gamma'][0]:.6f}",
                      delta=f"Error: {greeks['Gamma'][1]:.2e}")
        with col3:
            st.metric("Theta", f"{greeks['Theta'][0]:.6f}",
                      delta=f"Error: {greeks['Theta'][1]:.2e}")
        with col4:
            st.metric("Vega", f"{greeks['Vega'][0]:.6f}",
                      delta=f"Error: {greeks['Vega'][1]:.2e}")
        with col5:
            st.metric("Rho", f"{greeks['Rho'][0]:.6f}",
                      delta=f"Error: {greeks['Rho'][1]:.2e}")

        st.metric("Option Price", f"${option_price:.4f}")

        S_range = np.linspace(S_range_min, S_range_max, n_points)
        prices = [pricing_function({**base_params, 'S': s}) for s in
            S_range]

        deltas = []
        gammas = []
        thetas = []
        vegas = []
```

```
        rhos = []

        for s in S_range:
            d, _ = greek_calculator.compute_delta(s, use_richardson=
                use_richardson)
            g, _ = greek_calculator.compute_gamma(s, use_richardson=
                use_richardson)
            t, _ = greek_calculator.compute_theta(T, use_richardson=
                use_richardson)
            v, _ = greek_calculator.compute_vega(sigma, use_richardson=
                use_richardson)
            r_val, _ = greek_calculator.compute_rho(r, use_richardson=
                use_richardson)
            deltas.append(d)
            gammas.append(g)
            thetas.append(t)
            vegas.append(v)
            rhos.append(r_val)

        st.subheader("Price and Greeks Profiles")

        fig = make_subplots(
            rows=3, cols=2,
            subplot_titles=("Option Price", "Delta", "Gamma", "Theta",
                "Vega", "Rho"),
            vertical_spacing=0.08
        )

        fig.add_trace(go.Scatter(x=S_range, y=prices, mode='lines',
                                 name='Price', line=dict(color='#6865F2'
                                     , width=2)),
                      row=1, col=1)
        fig.add_vline(x=K, line_dash="dash", line_color="gray",
                      annotation_text="Strike", row=1, col=1)

        fig.add_trace(go.Scatter(x=S_range, y=deltas, mode='lines',
                                 name='Delta', line=dict(color='#5DFFBC'
                                     , width=2)),
                      row=1, col=2)

        fig.add_trace(go.Scatter(x=S_range, y=gammas, mode='lines',
                                 name='Gamma', line=dict(color='#FF6B6B'
                                     , width=2)),
                      row=2, col=1)

        fig.add_trace(go.Scatter(x=S_range, y=thetas, mode='lines',
                                 name='Theta', line=dict(color='#FFD93D'
                                     , width=2)),
                      row=2, col=2)

        fig.add_trace(go.Scatter(x=S_range, y=vegas, mode='lines',
                                 name='Vega', line=dict(color='#6BCF7F',
                                      width=2)),
                      row=3, col=1)

        fig.add_trace(go.Scatter(x=S_range, y=rhos, mode='lines',
                                 name='Rho', line=dict(color='#4D96FF',
                                    width=2)),
```

```
                    row=3, col=2)

        fig.update_xaxes(title_text="Spot Price", row=3, col=1)
        fig.update_xaxes(title_text="Spot Price", row=3, col=2)
        fig.update_yaxes(title_text="Price", row=1, col=1)
        fig.update_yaxes(title_text="Delta", row=1, col=2)
        fig.update_yaxes(title_text="Gamma", row=2, col=1)
        fig.update_yaxes(title_text="Theta", row=2, col=2)
        fig.update_yaxes(title_text="Vega", row=3, col=1)
        fig.update_yaxes(title_text="Rho", row=3, col=2)

        fig.update_layout(height=900, showlegend=False)
        st.plotly_chart(fig, use_container_width=True)

        st.subheader("Convergence Analysis")
        step_sizes = np.logspace(-4, -1, 20)
        delta_standard = []
        delta_richardson = []

        for h in step_sizes:
            d_std, _ = greek_calculator.compute_delta(S, h,
               use_richardson=False)
            d_rich, _ = greek_calculator.compute_delta(S, h,
               use_richardson=True)
            delta_standard.append(d_std)
            delta_richardson.append(d_rich)

        analytical_delta = norm.cdf((np.log(S / K) + (r + 0.5 * sigma
           ** 2) * T) / (sigma * np.sqrt(T)))

        fig_conv = go.Figure()
        fig_conv.add_trace(go.Scatter(x=step_sizes, y=np.abs(np.array(
           delta_standard) - analytical_delta),
                                      mode='lines+markers', name='
                                         Standard FD',
                                      line=dict(color='#FF6B6B', width
                                         =2)))
        fig_conv.add_trace(go.Scatter(x=step_sizes, y=np.abs(np.array(
           delta_richardson) - analytical_delta),
                                      mode='lines+markers', name='
                                         Richardson Extrapolation',
                                      line=dict(color='#6865F2', width
                                         =2)))
        fig_conv.add_hline(y=0, line_dash="dash", line_color="gray")
        fig_conv.update_xaxes(type="log", title_text="Step Size (h)")
        fig_conv.update_yaxes(type="log", title_text="Absolute Error")
        fig_conv.update_layout(title="Delta Convergence: Standard vs
           Richardson",
                               height=400)
        st.plotly_chart(fig_conv, use_container_width=True)
```

This dashboard provides comprehensive visualisation of Greek profiles, convergence analysis, and error diagnostics, enabling quantitative analysts to validate and understand Greek computation behaviour.

Expected Outputs

- Accurate Greek estimates (Delta, Gamma, Theta, Vega, Rho) computed using finite-difference methods with Richardson extrapolation, achieving higher-order accuracy than standard approximations.
- Interactive visualisations showing Greek profiles across different spot prices, demonstrating how sensitivities vary with market conditions.
- Convergence analysis comparing standard finite-difference methods with Richardson-extrapolated estimates, illustrating error reduction and optimal step size selection.
- Error estimates for each Greek, enabling users to assess numerical accuracy and validate results against analytical benchmarks where available.
- Adaptive step sizing diagnostics showing how the algorithm automatically adjusts perturbation sizes to balance truncation and round-off errors.
- Grid-based Greek computation for PDE and Monte Carlo price grids, enabling efficient sensitivity analysis from pre-computed pricing results.
- Professional dashboard interface suitable for risk management, quantitative research, and derivatives trading applications.

Interview Questions

1. Explain the trade-off between truncation error and round-off error in finite-difference methods. How does Richardson extrapolation help resolve this trade-off, and what are its limitations?
2. Describe the adaptive step sizing algorithm. How would you modify it to handle cases where the pricing function is non-smooth or discontinuous (e.g., barrier options, digital options)?
3. Compare central differencing with forward and backward differences. Why is central differencing preferred for Greek computation, and when might forward/backward differences be necessary?
4. How does Richardson extrapolation achieve higher-order accuracy? Derive the extrapolation formula for a second-order method and explain the error cancellation mechanism.
5. Discuss the challenges of computing Greeks from Monte Carlo price estimates. How does the use of common random numbers help, and what additional considerations arise when applying Richardson extrapolation to Monte Carlo Greeks?
6. How would you extend this framework to compute cross-Greeks (e.g., $\frac{\partial^2 V}{\partial S \partial \sigma}$) and higher-order sensitivities? What are the computational costs and accuracy considerations?
7. Explain the relationship between step size, pricing function smoothness, and Greek accuracy. How would you detect when step sizes are too small (round-off errors dominate) versus too large (truncation errors dominate)?
8. How would you validate Greek estimates computed using this framework? Discuss benchmarking against analytical formulas, comparison with alternative numerical methods, and convergence testing procedures.

Resume Bullets

- Developed a robust finite-difference Greek computation framework implementing Richardson extrapolation and adaptive step sizing, achieving fourth-order accuracy and automatic error minimisation for production risk systems.
- Built an interactive Streamlit dashboard with real-time Greek profiling, convergence analysis, and error diagnostics, enabling quantitative analysts to validate sensitivity calculations across diverse market scenarios.
- Designed extensible software architecture supporting both analytical and numerical pricing functions, with grid-based Greek computation for PDE and Monte Carlo pricing engines, facilitating integration across derivative product lines.
- Implemented sophisticated numerical algorithms combining central differencing optimisation with Richardson extrapolation, reducing computational costs whilst maintaining high accuracy standards required for regulatory reporting and risk management.